

SIMATS ENGINEERING

CO2-ASSESSMENT TOOL 2

SCENARIO BASED

COURSE CODE: CSA0201

COURSE NAME: C Programming

Student Name: Shaik Mubarak

Reg No: 192425194

Question 1: Electric Vehicle Battery Manufacturing System

Title

Electric Vehicle Battery Manufacturing System

Problem Statement

Develop a C program to monitor monthly battery production, calculate annual production, average production, production efficiency, and determine incentive eligibility or inspection status based on the production target.

Problem Understanding & Context Analysis

The company aims to monitor battery production throughout the year. The program helps calculate production statistics, reward high-performing months, and identify months requiring inspection. It reduces manual calculations and improves production management.

Concepts Used in C Programming

- Variables and Data Types
- Arithmetic Operators
- Relational Operators
- if-else Statements
- for Loop
- break and continue
- Input/Output Functions

Requirement Analysis

Inputs

- Monthly production values
- Production target (5000 batteries)

Outputs

- Annual production
- Average production
- Production efficiency
- Incentive/Inspection status

Solution Design & Justification

The program uses a loop to process production for all twelve months. Arithmetic operators calculate totals and averages. Decision-making statements compare production with the target. The continue statement skips invalid entries, while break stops execution during plant shutdown. This approach is simple, efficient, and suitable for real-world production monitoring.

Algorithm

1. Start.
2. Set production target.
3. Read monthly production.

4. Skip invalid input.
5. Stop if shutdown occurs.
6. Calculate total and average.
7. Compare production with target.
8. Display results.
9. Stop.

Complete C Program

```
#include <stdio.h>
int main() {
    int production, total = 0, i;
    float average, efficiency;
    int target = 5000;
    for(i = 1; i <= 12; i++) {
        printf("Enter production for Month %d (-1 for shutdown): ", i);
        scanf("%d", &production);
        if(production == -1) {
            printf("Plant Shutdown!\n");
            break;
        }
        if(production < 0) {
            printf("Invalid Production. Skipping...\n");
            continue;
        }
        total += production;
        if(production > target)
            printf("Incentive Eligible\n");
        else if(production < 0.8 * target)
            printf("Inspection Required\n");
        else
            printf("Target Achieved\n");
    }
    average = total / 12.0;
    efficiency = (average / target) * 100;
    printf("\nAnnual Production = %d", total);
    printf("\nAverage Production = %.2f", average);
    printf("\nProduction Efficiency = %.2f%%", efficiency);
    return 0;
}
```

Output

Programiz
C Online Compiler

main.c

Run

Share

Output

```
1 #include <stdio.h>
2- int main() {
3     int production, total = 0, i;
4     float average, efficiency;
5     int target = 5000;
6-     for(i = 1; i <= 12; i++) {
7         printf("Enter production for Month %d (-1 for shutdown): ", i);
8         scanf("%d", &production);
9-         if(production == -1) {
10             printf("Plant Shutdown!\n");
11             break;
12         }
13-         if(production < 0) {
14             printf("Invalid Production. Skipping...\n");
15             continue;
16         }
17         total += production;
18         if(production > target)
19             printf("Incentive Eligible\n");
20         else if(production < 0.8 * target)
21             printf("Inspection Required\n");
22         else
23             printf("Target Achieved\n");
24     }
25     average = total / 12.0;
26     efficiency = (average / target) * 100;
27     printf("\nAnnual Production = %d", total);
28     printf("\nAverage Production = %.2f", average);
29     printf("\nProduction Efficiency = %.2f%%", efficiency);
30     return 0;
31 }
32
33
```

Enter production for Month 1 (-1 for shutdown): 5
Inspection Required
Enter production for Month 2 (-1 for shutdown): 2
Inspection Required
Enter production for Month 3 (-1 for shutdown): -1
Plant Shutdown!

Annual Production = 7
Average Production = 0.58
Production Efficiency = 0.01%

=== Code Execution Successful ===

Explanation of the Code

The program accepts monthly production, calculates annual and average production, checks target achievement using **if-else**, processes all months using a **for loop**, skips invalid inputs using **continue**, and terminates if the plant shuts down using **break**.

Applications

- Battery Manufacturing Industries
- Production Monitoring Systems
- Factory Automation
- Performance Analysis

Conclusion

The program automates production monitoring, improves accuracy, and helps management evaluate manufacturing performance efficiently.

Question 2: Smart Agriculture Irrigation Management System

Title

Smart Agriculture Irrigation Management System

Problem Statement

Develop a C program to monitor soil moisture levels of different agricultural fields. The program should calculate the average soil moisture, identify fields requiring irrigation, and determine water usage efficiency. It should also skip faulty sensor readings using **continue** and stop monitoring if the irrigation controller fails using **break**.

Problem Understanding & Context Analysis

Efficient irrigation is essential for improving crop productivity and conserving water resources. Farmers need a system to monitor soil moisture levels regularly and decide whether irrigation is required. Automating this process reduces water wastage and improves crop health. The program helps farmers analyze moisture levels, identify dry fields, and ensure efficient irrigation management.

Concepts Used in C Programming

- Variables and Data Types
- Arithmetic Operators

- Relational Operators
- if-else Statements
- for Loop
- break and continue
- Input/Output Functions

Requirement Analysis

Inputs

- Soil moisture percentage of each field
- Standard moisture level (40%)

Outputs

- Average soil moisture
- Irrigation status
- Water usage efficiency

Solution Design & Justification

The program reads soil moisture values for all fields using a loop. It calculates the average moisture level and compares each value with the standard moisture level. Fields with low moisture require irrigation, while invalid sensor readings are ignored using continue. If the irrigation controller fails, the program terminates using break. This approach improves irrigation planning and water conservation.

Algorithm

1. Start.
2. Set standard moisture level.
3. Read moisture values.
4. Skip invalid readings.
5. Stop if controller failure occurs.
6. Calculate average moisture.
7. Compare with standard level.
8. Display irrigation status.
9. Stop.

Complete C Program

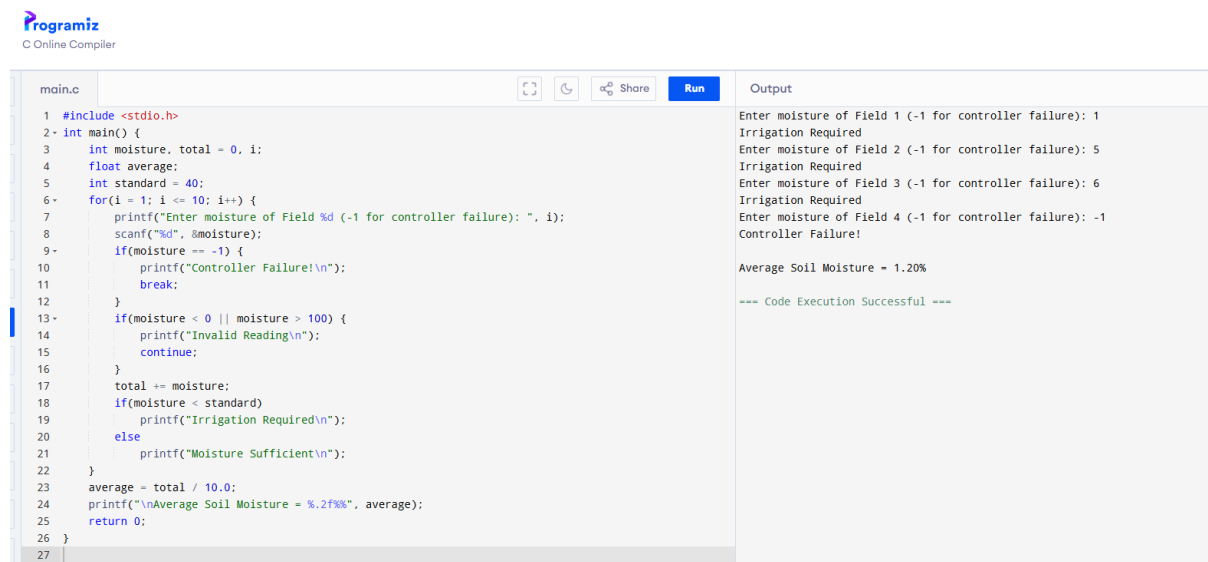
```
#include <stdio.h>
int main() {
    int moisture, total = 0, i;
    float average;
    int standard = 40;
    for(i = 1; i <= 10; i++) {
        printf("Enter moisture of Field %d (-1 for controller failure): ", i);
        scanf("%d", &moisture);
        if(moisture == -1) {
            printf("Controller Failure!\n");
            break;
        }
        if(moisture < 0 || moisture > 100) {
            printf("Invalid Reading\n");
            continue;
        }
        total += moisture;
    }
    average = total / i;
    printf("Average moisture level: %.2f\n", average);
    if(average < 40) {
        printf("Irrigation required\n");
    } else {
        printf("No irrigation required\n");
    }
}
```

```

        total += moisture;
    if(moisture < standard)
        printf("Irrigation Required\n");
    else
        printf("Moisture Sufficient\n");
}
average = total / 10.0;
printf("\nAverage Soil Moisture = %.2f%%", average);
return 0;
}

```

Output



The screenshot shows the Programiz C Online Compiler interface. On the left, the code for `main.c` is displayed, and on the right, the output of the program is shown.

```

main.c
1 #include <stdio.h>
2 int main() {
3     int moisture, total = 0, i;
4     float average;
5     int standard = 40;
6     for(i = 1; i <= 10; i++) {
7         printf("Enter moisture of Field %d (-1 for controller failure): ", i);
8         scanf("%d", &moisture);
9         if(moisture == -1) {
10             printf("Controller Failure!\n");
11             break;
12         }
13         if(moisture < 0 || moisture > 100) {
14             printf("Invalid Reading\n");
15             continue;
16         }
17         total += moisture;
18         if(moisture < standard)
19             printf("Irrigation Required\n");
20         else
21             printf("Moisture Sufficient\n");
22     }
23     average = total / 10.0;
24     printf("\nAverage Soil Moisture = %.2f%%", average);
25     return 0;
26 }
27

```

Output

```

Enter moisture of Field 1 (-1 for controller failure): 1
Irrigation Required
Enter moisture of Field 2 (-1 for controller failure): 5
Irrigation Required
Enter moisture of Field 3 (-1 for controller failure): 6
Irrigation Required
Enter moisture of Field 4 (-1 for controller failure): -1
Controller Failure!

Average Soil Moisture = 1.20%

=== Code Execution Successful ===

```

Explanation of the Code

The program accepts soil moisture values for ten fields. A for loop processes each field. The continue statement skips invalid readings, while break terminates the program if the controller fails. The program calculates the average moisture and determines whether irrigation is required based on the standard moisture level.

Applications

- Smart Farming
- Precision Agriculture
- Automated Irrigation Systems
- Water Resource Management

Conclusion

The Smart Agriculture Irrigation Management System helps farmers monitor soil moisture efficiently and optimize water usage. The program automates irrigation decisions, improves crop management, and demonstrates the practical application of C programming in modern agriculture.

Question 3: Smart Hospital Patient Monitoring System

Title

Smart Hospital Patient Monitoring System

Problem Statement

Develop a C program to monitor patients' body temperatures in a hospital. The program should calculate the average temperature, identify patients with fever, and generate alerts for critical conditions. Invalid temperature readings should be ignored using continue, and monitoring should stop if the monitoring device fails using break.

Problem Understanding & Context Analysis

Hospitals continuously monitor patients' vital signs to provide timely medical care. Body temperature is an important health indicator that helps detect fever or critical illness. Automating temperature monitoring enables healthcare professionals to identify abnormal conditions quickly, reducing manual effort and improving patient safety.

Concepts Used in C Programming

- Variables and Data Types
- Arithmetic Operators
- Relational Operators
- if-else Statements
- for Loop
- break and continue
- Input/Output Functions

Requirement Analysis

Inputs

- Patient temperature readings
- Normal temperature limit (37°C)

Outputs

- Average temperature
- Fever status
- Critical condition alert

Solution Design & Justification

The program records temperature readings for multiple patients using a loop. It calculates the average temperature and compares each reading with the normal body temperature. Invalid readings are ignored using continue, while monitoring stops immediately if the monitoring device fails using break. This solution improves patient monitoring and supports quick medical decisions.

Algorithm

1. Start.
2. Set normal temperature.
3. Read patient temperatures.
4. Ignore invalid readings.
5. Stop if device failure occurs.
6. Calculate average temperature.
7. Identify fever and critical cases.

8. Display results.
9. Stop.

Complete C Program

```
#include <stdio.h>
int main() {
    float temp, total = 0, average;
    int i;
    for(i = 1; i <= 10; i++) {
        printf("Enter temperature of Patient %d (-1 to stop): ", i);
        scanf("%f", &temp);
        if(temp == -1) {
            printf("Monitoring Device Failure!\n");
            break;
        }
        if(temp < 30 || temp > 45) {
            printf("Invalid Temperature\n");
            continue;
        }
        total += temp;
        if(temp > 39)
            printf("Critical Alert\n");
        else if(temp > 37)
            printf("Fever Detected\n");
        else
            printf("Normal\n");
    }
    average = total / 10;
    printf("\nAverage Temperature = %.2f°C", average);
    return 0;
}
```

Output

 Programiz
C Online Compiler

main.c	Output
<pre>1 #include <stdio.h> 2 int main() { 3 float temp, total = 0, average; 4 int i; 5 for(i = 1; i <= 10; i++) { 6 printf("Enter temperature of Patient %d (-1 to stop): ", i); 7 scanf("%f", &temp); 8 if(temp == -1) { 9 printf("Monitoring Device Failure!\n"); 10 break; 11 } 12 if(temp < 30 temp > 45) { 13 printf("Invalid Temperature\n"); 14 continue; 15 } 16 total += temp; 17 if(temp > 39) 18 printf("Critical Alert\n"); 19 else if(temp > 37) 20 printf("Fever Detected\n"); 21 else 22 printf("Normal\n"); 23 } 24 average = total / 10; 25 printf("\nAverage Temperature = %.2f°C", average); 26 return 0; 27 } 28</pre>	<pre>Enter temperature of Patient 1 (-1 to stop): 1 Invalid Temperature Enter temperature of Patient 2 (-1 to stop): 5 Invalid Temperature Enter temperature of Patient 3 (-1 to stop): 6 Invalid Temperature Enter temperature of Patient 4 (-1 to stop): -1 Monitoring Device Failure! Average Temperature = 0.00°C --- Code Execution Successful ---</pre>

Explanation of the Code

The program accepts patient temperature readings using a for loop. It validates each reading, ignores invalid values using continue, and terminates monitoring if the device fails using break. The program calculates the average temperature and identifies patients with fever or critical conditions.

Applications

- Hospitals
- Intensive Care Units (ICU)
- Patient Health Monitoring
- Smart Healthcare Systems

Conclusion

The Smart Hospital Patient Monitoring System automates temperature analysis and assists healthcare professionals in identifying abnormal health conditions. It demonstrates how C programming can be used to improve patient care through efficient monitoring and decision-making.

Question 4: Smart Traffic Management System

Title

Smart Traffic Management System

Problem Statement

Develop a C program to monitor vehicle count at a traffic signal, calculate total traffic flow, identify congestion levels, and manage traffic efficiently.

Problem Understanding & Context Analysis

Traffic congestion causes delays, fuel wastage, and pollution. A smart traffic management system helps monitor vehicle flow and identify congested roads for better traffic control.

Concepts Used in C Programming

- Variables and Data Types
- Arithmetic Operators
- Relational Operators
- if-else Statements
- for Loop
- break and continue
- Input/Output Functions

Requirement Analysis

Inputs

- Number of vehicles at each signal

Outputs

- Total vehicles
- Average traffic
- Congestion status

Solution Design & Justification

The program reads vehicle count from different traffic signals. It calculates total traffic and checks congestion levels. Invalid values are skipped using continue, and processing stops if the traffic monitoring system fails using break.

Algorithm

1. Start

2. Read vehicle count
3. Ignore invalid values
4. Stop if system fails
5. Calculate total and average
6. Check congestion level
7. Display results
8. Stop

Complete C Program

```
#include <stdio.h>
int main() {
    int vehicles, total = 0, i;

    for(i = 1; i <= 10; i++) {
        printf("Enter vehicle count at Signal %d (-1 to stop): ", i);
        scanf("%d", &vehicles);
        if(vehicles == -1)
            break;
        if(vehicles < 0)
            continue;
        total += vehicles;
        if(vehicles > 100)
            printf("Heavy Traffic\n");
        else
            printf("Normal Traffic\n");
    }
    printf("Total Vehicles = %d\n", total);
    return 0;
}
```

Output

Programiz
C Online Compiler

main.c	Output
<pre>1 #include <stdio.h> 2 int main() { 3 int vehicles, total = 0, i; 4 5 for(i = 1; i <= 10; i++) { 6 printf("Enter vehicle count at Signal %d (-1 to stop): ", i); 7 scanf("%d", &vehicles); 8 if(vehicles == -1) 9 break; 10 if(vehicles < 0) 11 continue; 12 total += vehicles; 13 if(vehicles > 100) 14 printf("Heavy Traffic\n"); 15 else 16 printf("Normal Traffic\n"); 17 } 18 printf("Total Vehicles = %d\n", total); 19 return 0; 20 }</pre>	<pre>Enter vehicle count at Signal 1 (-1 to stop): 5 Normal Traffic Enter vehicle count at Signal 2 (-1 to stop): 9 Normal Traffic Enter vehicle count at Signal 3 (-1 to stop): 3 Normal Traffic Enter vehicle count at Signal 4 (-1 to stop): 2 Normal Traffic Enter vehicle count at Signal 5 (-1 to stop): -1 Total Vehicles = 19 === Code Execution Successful ===</pre>

Explanation of the Code

The program records vehicle count, calculates total traffic, and identifies congestion using conditional statements.

Applications

- Smart Cities
- Traffic Monitoring

- Road Safety Systems

Conclusion

The program helps traffic authorities monitor congestion and improve traffic flow.

Question 5: Online Retail Sales Monitoring System

Title

Online Retail Sales Monitoring System

Problem Statement

Develop a C program to monitor daily sales, calculate total revenue, and identify high-performing sales days.

Problem Understanding & Context Analysis

Retail companies analyze daily sales to evaluate business performance and improve marketing strategies. Automated monitoring helps generate accurate reports.

Concepts Used in C Programming

- Variables
- Arithmetic Operators
- if-else
- for Loop
- break
- continue
- Input/Output

Requirement Analysis

Inputs

- Daily sales amount

Outputs

- Total sales
- Average sales
- High sales status

Solution Design & Justification

The program processes daily sales using loops. Invalid sales entries are skipped, while processing stops if the system shuts down.

Algorithm

1. Start
2. Read daily sales
3. Ignore invalid entries
4. Stop on shutdown
5. Calculate total
6. Display results
7. Stop

Complete C Program

```
#include <stdio.h>
```

```
int main() {
    float sale, total = 0;
    int i;

    for(i = 1; i <= 7; i++) {
        printf("Enter Day %d Sales (-1 to stop): ", i);
        scanf("%f", &sale);
```

```

        if(sale == -1)
            break;

        if(sale < 0)
            continue;

        total += sale;

        if(sale > 10000)
            printf("Excellent Sales\n");
    }

    printf("Total Sales = %.2f\n", total);

    return 0;
}

```

Output

Programiz
C Online Compiler

main.c	Output
<pre> 1 #include <stdio.h> 2 3- int main() { 4 float sale, total = 0; 5 int i; 6 7- for(i = 1; i <= 7; i++) { 8 printf("Enter Day %d Sales (-1 to stop): ", i); 9 scanf("%f", &sale); 10 11 if(sale == -1) 12 break; 13 14 if(sale < 0) 15 continue; 16 17 total += sale; 18 19 if(sale > 10000) 20 printf("Excellent Sales\n"); 21 } 22 23 printf("Total Sales = %.2f\n", total); 24 25 return 0; 26 } 27 </pre>	<pre> Enter Day 1 Sales (-1 to stop): 5 Enter Day 2 Sales (-1 to stop): 6 Enter Day 3 Sales (-1 to stop): 1 Enter Day 4 Sales (-1 to stop): 9 Enter Day 5 Sales (-1 to stop): 64 Enter Day 6 Sales (-1 to stop): -4 Enter Day 7 Sales (-1 to stop): -1 Total Sales = 85.00 === Code Execution Successful === </pre>

Explanation of the Code

The program calculates total sales and identifies days with excellent sales performance.

Applications

- Retail Stores
- E-Commerce
- Business Analytics

Conclusion

The system simplifies sales monitoring and improves business decision-making.

Question 6: Smart Energy Consumption Monitoring System

Title

Smart Energy Consumption Monitoring System

Problem Statement

Develop a C program to monitor electricity consumption, calculate total energy usage, and identify high-consumption households.

Problem Understanding & Context Analysis

Monitoring electricity usage helps consumers reduce energy wastage and lower electricity bills. The system identifies excessive energy consumption for better resource management.

Concepts Used in C Programming

- Variables
- Arithmetic Operators
- Relational Operators
- if-else
- for Loop
- break
- continue

Requirement Analysis

Inputs

- Monthly energy consumption

Outputs

- Total consumption
- Average consumption
- High usage status

Solution Design & Justification

The program reads monthly energy usage, calculates totals, and compares consumption with the standard limit. Invalid values are ignored, and monitoring stops if the meter fails.

Algorithm

1. Start
2. Read energy usage
3. Skip invalid values
4. Stop if meter fails
5. Calculate total
6. Display report
7. Stop

Complete C Program

```
#include <stdio.h>
int main() {
    int units, total = 0, i;
    for(i = 1; i <= 12; i++) {
        printf("Enter Units for Month %d (-1 to stop): ", i);
        scanf("%d", &units);
        if(units == -1)
            break;
        if(units < 0)
            continue;
        total += units;
        if(units > 500)
            printf("High Consumption\n");
    }

    printf("Total Units = %d\n", total);
    return 0;
}
```

}
Output


C Online Compiler

main.c	Output
<pre>1 #include <stdio.h> 2 int main() { 3 int units, total = 0, i; 4 for(i = 1; i <= 12; i++) { 5 printf("Enter Units for Month %d (-1 to stop): ", i); 6 scanf("%d", &units); 7 if(units == -1) 8 break; 9 if(units < 0) 10 continue; 11 total += units; 12 if(units > 500) 13 printf("High Consumption\n"); 14 } 15 16 printf("Total Units = %d\n", total); 17 return 0; 18 } 19</pre>	<pre>Enter Units for Month 1 (-1 to stop): 5 Enter Units for Month 2 (-1 to stop): 6 Enter Units for Month 3 (-1 to stop): 7 Enter Units for Month 4 (-1 to stop): 5 Enter Units for Month 5 (-1 to stop): -1 Total Units = 23 === Code Execution Successful ===</pre>

Explanation of the Code

The program processes monthly electricity consumption, calculates the total usage, and identifies high-energy consumers.

Applications

- Smart Homes
- Electricity Boards
- Energy Management Systems

Conclusion

The program helps monitor electricity usage efficiently and promotes energy conservation

Question 7: Smart Water Supply Monitoring System

Title

Smart Water Supply Monitoring System

Problem Statement

Develop a C program to monitor daily water supply, calculate total water distributed, and identify areas with insufficient water supply.

Problem Understanding & Context Analysis

Water is a valuable resource, and its proper distribution is essential. This system helps monitor daily water supply, identify shortages, and improve water management.

Concepts Used in C Programming

- Variables and Data Types
- Arithmetic Operators
- Relational Operators
- if-else Statements
- for Loop
- break and continue
- Input/Output Functions

Requirement Analysis

Inputs

- Daily water supply (liters)

Outputs

- Total water supplied

- Average supply
- Supply status

Solution Design & Justification

The program reads daily water supply, calculates the total and average, and compares each day's supply with the required limit. Invalid inputs are skipped using continue, and monitoring stops if the water system fails using break.

Algorithm

1. Start
2. Read daily water supply
3. Skip invalid values
4. Stop if system fails
5. Calculate total and average
6. Display supply status
7. Stop

Complete C Program

```
#include <stdio.h>
int main() {
    int water, total = 0, i;
    for(i = 1; i <= 7; i++) {
        printf("Enter water supplied on Day %d (-1 to stop): ", i);
        scanf("%d", &water);
        if(water == -1)
            break;
        if(water < 0)
            continue;
        total += water;
        if(water < 1000)
            printf("Low Water Supply\n");
        else
            printf("Adequate Water Supply\n");
    }
    printf("Total Water Supplied = %d Liters\n", total);
    return 0;
}
```

Output

Programiz
C Online Compiler

Program

main.c	Output
<pre>1 #include <stdio.h> 2 int main() { 3 int water, total = 0, i; 4 for(i = 1; i <= 7; i++) { 5 printf("Enter water supplied on Day %d (-1 to stop): ", i); 6 scanf("%d", &water); 7 if(water == -1) 8 break; 9 if(water < 0) 10 continue; 11 total += water; 12 if(water < 1000) 13 printf("Low Water Supply\n"); 14 else 15 printf("Adequate Water Supply\n"); 16 } 17 printf("Total Water Supplied = %d Liters\n", total); 18 return 0; 19 } 20</pre>	<pre>Enter water supplied on Day 1 (-1 to stop): 5 Low Water Supply Enter water supplied on Day 2 (-1 to stop): 9 Low Water Supply Enter water supplied on Day 3 (-1 to stop): 6 Low Water Supply Enter water supplied on Day 4 (-1 to stop): 3 Low Water Supply Enter water supplied on Day 5 (-1 to stop): -1 Total Water Supplied = 23 Liters === Code Execution Successful ===</pre>

Explanation of the Code

The program reads daily water supply, calculates the total supply, and identifies days with insufficient water using conditional statements.

Applications

- Municipal Water Supply
- Smart Water Distribution
- Water Resource Management

Conclusion

The program helps monitor water distribution and supports efficient water resource management.

Question 8: Smart Library Book Management System

Title

Smart Library Book Management System

Problem Statement

Develop a C program to monitor book borrowing, calculate total books issued, and identify overdue returns.

Problem Understanding & Context Analysis

Libraries need an efficient system to monitor book transactions and ensure timely returns. Automation simplifies record management and improves library services.

Concepts Used in C Programming

- Variables
- Arithmetic Operators
- Relational Operators
- if-else
- for Loop
- break
- continue
- Input/Output Functions

Requirement Analysis

Inputs

- Number of books borrowed
- Days overdue

Outputs

- Total books issued
- Return status
- Overdue status

Solution Design & Justification

The program records book issue details, calculates total books issued, and checks overdue returns. Invalid entries are skipped, and processing stops if the library system fails.

Algorithm

1. Start
2. Read borrowing details
3. Ignore invalid data
4. Stop if system fails
5. Calculate totals
6. Display overdue status
7. Stop

Complete C Program

```
#include <stdio.h>
int main() {
    int books, days, total = 0, i;
    for(i = 1; i <= 5; i++) {
        printf("Enter books borrowed (-1 to stop): ");
        scanf("%d", &books);
        if(books == -1)
            break;
        if(books < 0)
            continue;
        printf("Enter overdue days: ");
        scanf("%d", &days);
        total += books;
        if(days > 15)
            printf("Overdue Return\n");
        else
            printf("Returned on Time\n");
    }
    printf("Total Books Issued = %d\n", total);
    return 0;
}
```

Output

main.c	Output
<pre>1 #include <stdio.h> 2 int main() { 3 int books, days, total = 0, i; 4 for(i = 1; i <= 5; i++) { 5 printf("Enter books borrowed (-1 to stop): "); 6 scanf("%d", &books); 7 if(books == -1) 8 break; 9 if(books < 0) 10 continue; 11 printf("Enter overdue days: "); 12 scanf("%d", &days); 13 total += books; 14 if(days > 15) 15 printf("Overdue Return\n"); 16 else 17 printf("Returned on Time\n"); 18 } 19 printf("Total Books Issued = %d\n", total); 20 21 return 0; 22 } 23</pre>	<pre>Enter books borrowed (-1 to stop): 6 Enter overdue days: 3 Returned on Time Enter books borrowed (-1 to stop): 2 Enter overdue days: 9 Returned on Time Enter books borrowed (-1 to stop): 61 Enter overdue days: -1 Returned on Time Enter books borrowed (-1 to stop): -1 Total Books Issued = 69 === Code Execution Successful ===</pre>

Explanation of the Code

The program monitors book transactions, calculates total issued books, and identifies overdue returns.

Applications

- Libraries
- Educational Institutions
- Digital Library Systems

Conclusion

The system improves library management and ensures efficient tracking of borrowed books.

Question 9: Smart Banking Transaction Monitoring System

Title

Smart Banking Transaction Monitoring System

Problem Statement

Develop a C program to monitor banking transactions, calculate total transaction amount, and identify suspicious transactions.

Problem Understanding & Context Analysis

Banks process thousands of transactions daily. Monitoring transaction amounts helps detect suspicious activities and improves financial security.

Concepts Used in C Programming

- Variables
- Arithmetic Operators
- Relational Operators
- if-else
- for Loop
- break
- continue
- Input/Output Functions

Requirement Analysis

Inputs

- Transaction amount

Outputs

- Total transaction amount
- Transaction status
- Suspicious transaction alert

Solution Design & Justification

The program records transaction amounts, calculates the total value, and flags transactions exceeding a specified limit. Invalid values are ignored, and processing stops if the banking server fails.

Algorithm

1. Start
2. Read transaction amount
3. Skip invalid entries
4. Stop if server fails
5. Calculate total amount
6. Display transaction status
7. Stop

Complete C Program

```
#include <stdio.h>
int main() {
    float amount, total = 0;
    int i;
    for(i = 1; i <= 10; i++) {
        printf("Enter transaction amount (-1 to stop): ");
        scanf("%f", &amount);
        if(amount == -1)
            break;
        if(amount < 0)
            continue;
        total += amount;
        if(amount > 50000)
            printf("Suspicious Transaction\n");
    }
}
```

```

        else
            printf("Transaction Accepted\n");
    }
    printf("Total Transaction Amount = %.2f\n", total);
    return 0;
}

```

Output



main.c	Output
<pre> 1 #include <stdio.h> 2 int main() { 3 float amount, total = 0; 4 int i; 5 for(i = 1; i <= 10; i++) { 6 printf("Enter transaction amount (-1 to stop): "); 7 scanf("%f", &amount); 8 if(amount == -1) 9 break; 10 if(amount < 0) 11 continue; 12 total += amount; 13 if(amount > 50000) 14 printf("Suspicious Transaction\n"); 15 else 16 printf("Transaction Accepted\n"); 17 } 18 printf("Total Transaction Amount = %.2f\n", total); 19 return 0; 20 } 21 22 </pre>	<pre> Enter transaction amount (-1 to stop): 5 Transaction Accepted Enter transaction amount (-1 to stop): 9 Transaction Accepted Enter transaction amount (-1 to stop): 3 Transaction Accepted Enter transaction amount (-1 to stop): -1 Total Transaction Amount = 17.00 === Code Execution Successful === </pre>

Explanation of the Code

The program processes transaction amounts, calculates the total value, and identifies suspicious transactions using conditional statements.

Applications

- Banking Systems
- Financial Monitoring
- Fraud Detection

Conclusion

The program improves banking security by monitoring transactions and identifying unusual financial activities.

Question 10: Smart Warehouse Inventory Management System

Title

Smart Warehouse Inventory Management System

Problem Statement

Develop a C program to monitor warehouse inventory, calculate total stock, identify low-stock items, and generate reorder alerts.

Problem Understanding & Context Analysis

Inventory management is essential to ensure products are always available. This system helps monitor stock levels, prevent shortages, and improve warehouse operations.

Concepts Used in C Programming

- Variables and Data Types
- Arithmetic Operators
- Relational Operators

- if-else Statements
- for Loop
- break and continue
- Input/Output Functions

Requirement Analysis

Inputs

- Stock quantity of products

Outputs

- Total stock
- Average stock
- Reorder status

Solution Design & Justification

The program reads stock quantities, calculates the total inventory, and compares stock with the minimum threshold. Invalid entries are skipped using continue, while processing stops if the warehouse system fails using break.

Algorithm

1. Start
2. Read stock quantity
3. Skip invalid values
4. Stop if system fails
5. Calculate total stock
6. Check reorder level
7. Display results
8. Stop

Complete C Program

```
#include <stdio.h>
int main() {
    int stock, total = 0, i;
    for(i = 1; i <= 10; i++) {
        printf("Enter stock of Product %d (-1 to stop): ", i);
        scanf("%d", &stock);
        if(stock == -1)
            break;
        if(stock < 0)
            continue;
        total += stock;
        if(stock < 20)
            printf("Reorder Required\n");
        else
            printf("Stock Available\n");
    }
    printf("Total Stock = %d\n", total);
    return 0;
}
```

Output



main.c	Output
<pre>1 #include <stdio.h> 2- int main() { 3 int stock, total = 0, i; 4+ for(i = 1; i <= 10; i++) { 5 printf("Enter stock of Product %d (-1 to stop): ", i); 6 scanf("%d", &stock); 7 if(stock == -1) 8 break; 9 if(stock < 0) 10 continue; 11 total += stock; 12 if(stock < 20) 13 printf("Reorder Required\n"); 14 else 15 printf("Stock Available\n"); 16 } 17 printf("Total Stock = %d\n", total); 18 return 0; 19 } 20</pre>	<pre>Enter stock of Product 1 (-1 to stop): 5 Reorder Required Enter stock of Product 2 (-1 to stop): 4 Reorder Required Enter stock of Product 3 (-1 to stop): 2 Reorder Required Enter stock of Product 4 (-1 to stop): -1 Total Stock = 11 === Code Execution Successful ===</pre>

Explanation of the Code

The program processes stock quantities, calculates total inventory, and identifies products that require reordering.

Applications

- Warehouses
- Retail Stores
- Supply Chain Management

Conclusion

The program helps manage inventory efficiently and prevents stock shortages.

Question 11: Smart Weather Monitoring System

Title

Smart Weather Monitoring System

Problem Statement

Develop a C program to monitor daily temperature, calculate average temperature, and identify extreme weather conditions.

Problem Understanding & Context Analysis

Weather monitoring helps predict climate changes and supports agriculture, disaster management, and public safety.

Concepts Used in C Programming

- Variables
- Arithmetic Operators
- Relational Operators
- if-else
- for Loop
- break
- continue
- Input/Output Functions

Requirement Analysis

Inputs

- Daily temperature

Outputs

- Average temperature

- Weather status

Solution Design & Justification

The program accepts daily temperature values, calculates the average, and identifies high-temperature days. Invalid readings are ignored using continue, and monitoring stops if the weather station fails.

Algorithm

1. Start
2. Read temperature
3. Skip invalid readings
4. Stop if station fails
5. Calculate average
6. Display weather status
7. Stop

Complete C Program

```
#include <stdio.h>

int main() {
    float temp, total = 0;
    int i;
    for(i = 1; i <= 7; i++) {
        printf("Enter temperature (-1 to stop): ");
        scanf("%f", &temp);
        if(temp == -1)
            break;
        if(temp < -20 || temp > 60)
            continue;
        total += temp;
        if(temp > 40)
            printf("Heat Alert\n");
        else
            printf("Normal Weather\n");
    }
    printf("Average Temperature = %.2f\n", total / 7);
    return 0;
}
```

Output

 Programiz
C Online Compiler

main.c	Output
<pre>1 #include <stdio.h> 2 int main() { 3 float temp, total = 0; 4 int i; 5 for(i = 1; i <= 7; i++) { 6 printf("Enter temperature (-1 to stop): "); 7 scanf("%f", &temp); 8 if(temp == -1) 9 break; 10 if(temp < -20 temp > 60) 11 continue; 12 total += temp; 13 if(temp > 40) 14 printf("Heat Alert\n"); 15 else 16 printf("Normal Weather\n"); 17 } 18 printf("Average Temperature = %.2f\n", total / 7); 19 return 0; 20 } 21</pre>	<pre>Enter temperature (-1 to stop): 9 Normal Weather Enter temperature (-1 to stop): 6 Normal Weather Enter temperature (-1 to stop): 3 Normal Weather Enter temperature (-1 to stop): 2 Normal Weather Enter temperature (-1 to stop): -1 Average Temperature = 2.86 === Code Execution Successful ===</pre>

Explanation of the Code

The program records temperature values, calculates the average, and identifies extreme weather conditions.

Applications

- Weather Forecasting
- Agriculture
- Disaster Management

Conclusion

The program improves weather monitoring and supports environmental analysis.

Question 12: Smart Student Attendance Management System

Title

Smart Student Attendance Management System

Problem Statement

Develop a C program to monitor student attendance, calculate attendance percentage, and determine eligibility for examinations.

Problem Understanding & Context Analysis

Educational institutions monitor attendance to ensure students meet the required attendance criteria. This system automates attendance calculations and eligibility checking.

Concepts Used in C Programming

- Variables
- Arithmetic Operators
- Relational Operators
- if-else
- for Loop
- break
- continue
- Input/Output Functions

Requirement Analysis

Inputs

- Number of classes attended
- Total classes conducted

Outputs

- Attendance percentage
- Eligibility status

Solution Design & Justification

The program calculates attendance percentage using arithmetic operators and determines eligibility using conditional statements. Invalid data is ignored, and processing stops if the attendance server fails.

Algorithm

1. Start
2. Read attendance details
3. Skip invalid entries
4. Stop if server fails
5. Calculate percentage
6. Check eligibility
7. Display results
8. Stop

Complete C Program

```
#include <stdio.h>
```

```

int main() {
    int attended, total;
    float percentage;
    printf("Enter classes attended: ");
    scanf("%d", &attended);
    if(attended == -1) {
        printf("Server Failure");
        return 0;
    }
    printf("Enter total classes: ");
    scanf("%d", &total);
    if(attended < 0 || total <= 0) {
        printf("Invalid Input");
        return 0;
    }
    percentage = (attended * 100.0) / total;
    printf("Attendance = %.2f%%\n", percentage);
    if(percentage >= 75)
        printf("Eligible for Exam");
    else
        printf("Not Eligible");
    return 0;
}

```

Output



C Online Compiler

main.c	Output
<pre> 1 #include <stdio.h> 2- int main() { 3 int attended, total; 4 float percentage; 5 printf("Enter classes attended: "); 6 scanf("%d", &attended); 7- if(attended == -1) { 8 printf("Server Failure"); 9 return 0; 10 } 11 printf("Enter total classes: "); 12 scanf("%d", &total); 13- if(attended < 0 total <= 0) { 14 printf("Invalid Input"); 15 return 0; 16 } 17 percentage = (attended * 100.0) / total; 18 printf("Attendance = %.2f%%\n", percentage); 19 if(percentage >= 75) 20 printf("Eligible for Exam"); 21 else 22 printf("Not Eligible"); 23 return 0; 24 } 25 </pre>	<pre> Enter classes attended: 25 Enter total classes: 35 Attendance = 71.43% Not Eligible === Code Execution Successful === </pre>

Explanation of the Code

The program calculates attendance percentage and checks whether the student meets the minimum attendance requirement.

Applications

- Schools
- Colleges
- Online Learning Platforms

Conclusion

The program automates attendance management and helps institutions determine examination eligibility efficiently.

Question 13: Smart Railway Reservation System

Title

Smart Railway Reservation System

Problem Statement

Develop a C program to manage railway ticket bookings, calculate total tickets booked, and identify waiting list passengers.

Problem Understanding & Context Analysis

Railway reservation systems help passengers book tickets efficiently. The system should manage bookings, confirm seat availability, and maintain waiting lists when seats are full.

Concepts Used in C Programming

- Variables and Data Types
- Arithmetic Operators
- Relational Operators
- if-else Statements
- for Loop
- break and continue
- Input/Output Functions

Requirement Analysis

Inputs

- Number of tickets requested
- Available seats

Outputs

- Booking status
- Total tickets booked
- Waiting list status

Solution Design & Justification

The program checks seat availability before booking. Confirmed bookings are processed, while excess requests are moved to the waiting list. Invalid inputs are skipped using continue, and the system stops using break during server failure.

Algorithm

1. Start
2. Read ticket request
3. Check seat availability
4. Confirm booking or waiting list
5. Update available seats
6. Display booking status
7. Stop

Complete C Program

```
#include<stdio.h>
```

```
int main() {  
    int seats = 100, tickets, booked = 0;  
    while(1) {  
        printf("Enter tickets required (-1 to stop): ");  
        scanf("%d", &tickets);  
        if(tickets == -1)  
            break;
```

```

    if(tickets <= 0)
        continue;
    if(tickets <= seats) {
        seats -= tickets;
        booked += tickets;
        printf("Booking Confirmed\n");
    }
    else
        printf("Waiting List\n");
}
printf("Total Tickets Booked = %d", booked);
return 0;
}

```

Output

Programiz
C Online Compiler

<pre> main.c 1 #include<stdio.h> 2 int main() { 3 int seats = 100, tickets, booked = 0; 4 while(1) { 5 printf("Enter tickets required (-1 to stop): "); 6 scanf("%d", &tickets); 7 if(tickets == -1) 8 break; 9 if(tickets <= 0) 10 continue; 11 if(tickets <= seats) { 12 seats -= tickets; 13 booked += tickets; 14 printf("Booking Confirmed\n"); 15 } 16 else 17 printf("Waiting List\n"); 18 } 19 printf("Total Tickets Booked = %d", booked); 20 return 0; 21 } 22 </pre>	Output <pre> Enter tickets required (-1 to stop): 5 Booking Confirmed Enter tickets required (-1 to stop): 3 Booking Confirmed Enter tickets required (-1 to stop): 6 Booking Confirmed Enter tickets required (-1 to stop): 23 Booking Confirmed Enter tickets required (-1 to stop): -1 Total Tickets Booked = 37 === Code Execution Successful === </pre>
--	--

Explanation of the Code

The program accepts ticket requests, verifies seat availability, confirms bookings, and places excess passengers on the waiting list.

Applications

- Railway Reservation
- Online Ticket Booking
- Transport Management

Conclusion

The system simplifies railway reservation and efficiently manages seat allocation.

Question 14: Smart Parking Management System

Title

Smart Parking Management System

Problem Statement

Develop a C program to monitor parking slots, calculate occupied spaces, and identify available parking slots.

Problem Understanding & Context Analysis

Parking management systems help reduce traffic congestion by efficiently utilizing parking spaces and informing drivers about slot availability.

Concepts Used in C Programming

- Variables

- Arithmetic Operators
- Relational Operators
- if-else
- for Loop
- break
- continue
- Input/Output Functions

Requirement Analysis

Inputs

- Vehicles entering parking

Outputs

- Occupied slots
- Available slots
- Parking status

Solution Design & Justification

The program records parked vehicles, updates available slots, and stops when the parking lot becomes full or the system shuts down.

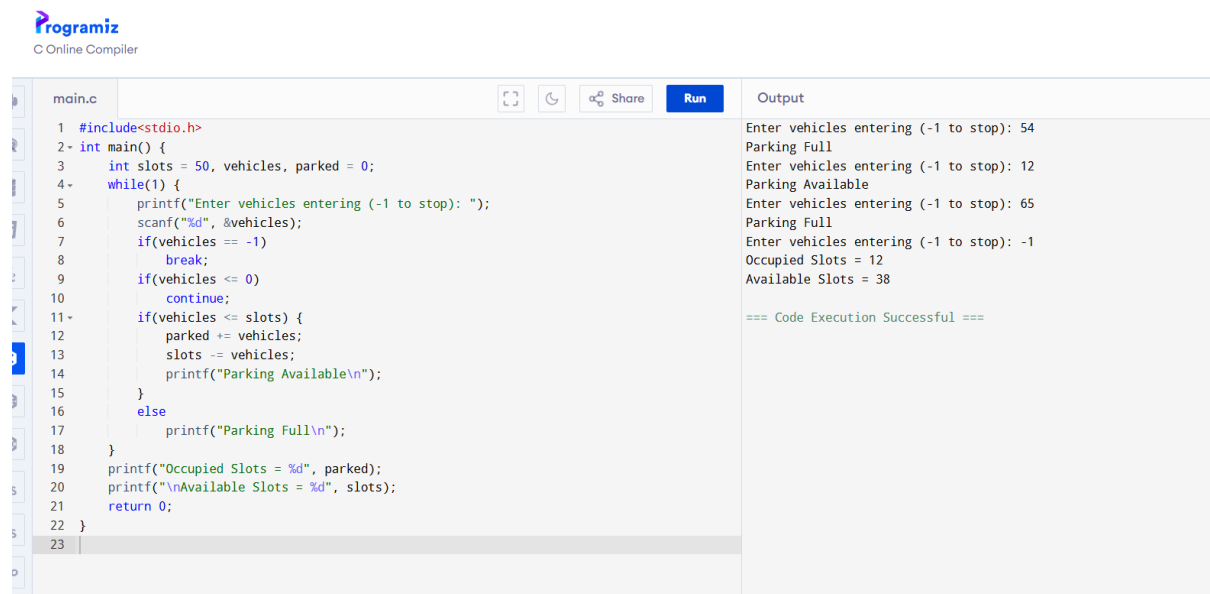
Algorithm

1. Start
2. Read vehicle entry
3. Check slot availability
4. Update parking count
5. Display parking status
6. Stop

Complete C Program

```
#include<stdio.h>
int main() {
    int slots = 50, vehicles, parked = 0;
    while(1) {
        printf("Enter vehicles entering (-1 to stop): ");
        scanf("%d", &vehicles);
        if(vehicles == -1)
            break;
        if(vehicles <= 0)
            continue;
        if(vehicles <= slots) {
            parked += vehicles;
            slots -= vehicles;
            printf("Parking Available\n");
        }
        else
            printf("Parking Full\n");
    }
    printf("Occupied Slots = %d", parked);
    printf("\n\nAvailable Slots = %d", slots);
    return 0;
}
```

Output



The screenshot shows the Programiz C Online Compiler interface. On the left, the code editor displays a C program named 'main.c'. The code initializes 50 parking slots and enters a loop where it prompts the user to enter a vehicle ID (-1 to stop). It checks if the slot is available and updates the count of occupied slots and available slots. The output window on the right shows the program's execution with sample inputs: 54, 12, 65, and -1. The final output indicates 12 occupied slots and 38 available slots, followed by a success message.

```
main.c
1 #include<stdio.h>
2- int main() {
3     int slots = 50, vehicles, parked = 0;
4-     while(1) {
5         printf("Enter vehicles entering (-1 to stop): ");
6         scanf("%d", &vehicles);
7         if(vehicles == -1)
8             break;
9         if(vehicles <= 0)
10            continue;
11-        if(vehicles <= slots) {
12            parked += vehicles;
13            slots -= vehicles;
14            printf("Parking Available\n");
15        }
16        else
17            printf("Parking Full\n");
18    }
19    printf("Occupied Slots = %d", parked);
20    printf("\n\nAvailable Slots = %d", slots);
21    return 0;
22 }
23
```

Output

```
Enter vehicles entering (-1 to stop): 54
Parking Full
Enter vehicles entering (-1 to stop): 12
Parking Available
Enter vehicles entering (-1 to stop): 65
Parking Full
Enter vehicles entering (-1 to stop): -1
Occupied Slots = 12
Available Slots = 38

=== Code Execution Successful ===
```

Explanation of the Code

The program updates occupied parking slots and checks whether additional vehicles can be accommodated.

Applications

- Shopping Malls
- Airports
- Smart Parking Systems

Conclusion

The program efficiently manages parking space utilization.

Question 15: Smart Water Quality Monitoring System

Title

Smart Water Quality Monitoring System

Problem Statement

Develop a C program to monitor water quality, calculate average pH level, and identify unsafe water samples.

Problem Understanding & Context Analysis

Water quality monitoring ensures safe drinking water by continuously checking pH levels and identifying contaminated samples.

Concepts Used in C Programming

- Variables
- Arithmetic Operators
- Relational Operators
- if-else
- for Loop
- break
- continue
- Input/Output Functions

Requirement Analysis

Inputs

- Water pH values

Outputs

- Average pH
- Water quality status

Solution Design & Justification

The program records pH values, calculates the average pH, and determines whether the water is safe. Invalid readings are skipped, and monitoring stops if the sensor fails.

Algorithm

1. Start
2. Read pH values
3. Skip invalid readings
4. Calculate average
5. Check water quality
6. Display results
7. Stop

Complete C Program

```
#include<stdio.h>
int main() {
    float ph, total = 0;
    int i;
    for(i = 1; i <= 5; i++) {
        printf("Enter pH value (-1 to stop): ");
        scanf("%f", &ph);
        if(ph == -1)
            break;
        if(ph < 0 || ph > 14)
            continue;
        total += ph;
        if(ph >= 6.5 && ph <= 8.5)
            printf("Safe Water\n");
        else
            printf("Unsafe Water\n");
    }
    printf("Average pH = %.2f", total / 5);
    return 0;
}
```

Output



main.c	Output
<pre>1 #include<stdio.h> 2 int main() { 3 float ph, total = 0; 4 int i; 5 for(i = 1; i <= 5; i++) { 6 printf("Enter pH value (-1 to stop): "); 7 scanf("%f", &ph); 8 if(ph == -1) 9 break; 10 if(ph < 0 ph > 14) 11 continue; 12 total += ph; 13 if(ph >= 6.5 && ph <= 8.5) 14 printf("Safe Water\n"); 15 else 16 printf("Unsafe Water\n"); 17 } 18 printf("Average pH = %.2f", total / 5); 19 return 0; 20 }</pre>	<pre>Enter pH value (-1 to stop): 45 Enter pH value (-1 to stop): 12 Unsafe Water Enter pH value (-1 to stop): 56 Enter pH value (-1 to stop): 6 Unsafe Water Enter pH value (-1 to stop): 7 Safe Water Average pH = 5.00 === Code Execution Successful ===</pre>

Explanation of the Code

The program records water pH values, calculates the average, and determines whether each sample is safe based on the standard pH range.

Applications

- Water Treatment Plants
- Environmental Monitoring
- Public Health

Conclusion

The system helps monitor water quality and ensures safe drinking water through automated analysis.

Question 16: Smart Online Examination System

Title

Smart Online Examination System

Problem Statement

Develop a C program to evaluate students' marks, calculate total and average marks, determine grades, and identify pass or fail status.

Problem Understanding & Context Analysis

Online examination systems simplify result processing by automatically evaluating marks and assigning grades. This reduces manual work and ensures accuracy.

Concepts Used in C Programming

- Variables and Data Types
- Arithmetic Operators
- Relational Operators
- if-else Statements
- for Loop
- break and continue
- Input/Output Functions

Requirement Analysis

Inputs

- Marks of students

Outputs

- Total marks
- Average marks
- Grade
- Pass/Fail status

Solution Design & Justification

The program accepts marks of students, calculates the total and average, assigns grades using conditional statements, skips invalid marks using continue, and stops processing if the examination server fails using break.

Algorithm

1. Start
2. Read student marks
3. Skip invalid marks
4. Stop if server fails
5. Calculate total and average
6. Assign grade
7. Display result
8. Stop

Complete C Program

```
#include <stdio.h>
int main() {
    int mark, total = 0, i;
    float average;
    for(i = 1; i <= 5; i++) {
        printf("Enter mark of Student %d (-1 to stop): ", i);
        scanf("%d", &mark);
        if(mark == -1)
            break;
        if(mark < 0 || mark > 100)
            continue;
        total += mark;
        if(mark >= 50)
            printf("Pass\n");
        else
            printf("Fail\n");
    }
    average = total / 5.0;
    printf("Total Marks = %d\n", total);
    printf("Average Marks = %.2f\n", average);
    return 0;
}
```

Output

Programiz
C Online Compiler

main.c	Output
<pre>1 #include <stdio.h> 2 int main() { 3 int mark, total = 0, i; 4 float average; 5 for(i = 1; i <= 5; i++) { 6 printf("Enter mark of Student %d (-1 to stop): ", i); 7 scanf("%d", &mark); 8 if(mark == -1) 9 break; 10 if(mark < 0 mark > 100) 11 continue; 12 total += mark; 13 if(mark >= 50) 14 printf("Pass\n"); 15 else 16 printf("Fail\n"); 17 } 18 average = total / 5.0; 19 printf("Total Marks = %d\n", total); 20 printf("Average Marks = %.2f\n", average); 21 return 0; 22 }</pre>	<pre>Enter mark of Student 1 (-1 to stop): 15 Fail Enter mark of Student 2 (-1 to stop): 85 Pass Enter mark of Student 3 (-1 to stop): 98 Pass Enter mark of Student 4 (-1 to stop): 75 Pass Enter mark of Student 5 (-1 to stop): -1 Total Marks = 273 Average Marks = 54.60 === Code Execution Successful ===</pre>

Explanation of the Code

The program accepts student marks, calculates the total and average, and determines pass or fail using conditional statements.

Applications

- Online Examination Systems
- Schools
- Colleges

Conclusion

The program automates result processing and improves accuracy in academic evaluation.

Question 17: Smart Air Quality Monitoring System

Title

Smart Air Quality Monitoring System

Problem Statement

Develop a C program to monitor Air Quality Index (AQI), calculate average AQI, and identify polluted areas.

Problem Understanding & Context Analysis

Monitoring air quality helps protect public health by identifying polluted regions and supporting environmental planning.

Concepts Used in C Programming

- Variables
- Arithmetic Operators
- Relational Operators
- if-else
- for Loop
- break
- continue
- Input/Output Functions

Requirement Analysis

Inputs

- AQI values

Outputs

- Average AQI
- Air quality status

Solution Design & Justification

The program reads AQI values, calculates the average AQI, identifies polluted areas, ignores invalid values using continue, and stops monitoring if the sensor fails using break.

Algorithm

1. Start
2. Read AQI values
3. Skip invalid values
4. Stop if sensor fails
5. Calculate average AQI
6. Display air quality
7. Stop

Complete C Program

```
#include <stdio.h>
int main() {
    int aqi, total = 0, i;
    for(i = 1; i <= 7; i++) {
        printf("Enter AQI (-1 to stop): ");
        scanf("%d", &aqi);
        if(aqi == -1)
            break;
        if(aqi < 0)
            continue;
        total += aqi;
        if(aqi > 150)
            printf("Poor Air Quality\n");
        else
            printf("Good Air Quality\n");
    }
    printf("Average AQI = %.2f\n", total / 7.0);
    return 0;
}
```

Output



main.c	Output
<pre>1 #include <stdio.h> 2 int main() { 3 int aqi, total = 0, i; 4 for(i = 1; i <= 7; i++) { 5 printf("Enter AQI (-1 to stop): "); 6 scanf("%d", &aqi); 7 if(aqi == -1) 8 break; 9 if(aqi < 0) 10 continue; 11 total += aqi; 12 if(aqi > 150) 13 printf("Poor Air Quality\n"); 14 else 15 printf("Good Air Quality\n"); 16 } 17 printf("Average AQI = %.2f\n", total / 7.0); 18 return 0; 19 } 20</pre>	<pre>Enter AQI (-1 to stop): 45 Good Air Quality Enter AQI (-1 to stop): 19 Good Air Quality Enter AQI (-1 to stop): 89 Good Air Quality Enter AQI (-1 to stop): 15 Good Air Quality Enter AQI (-1 to stop): -1 Average AQI = 24.00 === Code Execution Successful ===</pre>

Explanation of the Code

The program processes AQI values, calculates the average, and identifies polluted areas.

Applications

- Environmental Monitoring
- Smart Cities
- Pollution Control Boards

Conclusion

The system helps monitor air quality and supports environmental protection.

Question 18: Smart Hospital Medicine Inventory System

Title

Smart Hospital Medicine Inventory System

Problem Statement

Develop a C program to monitor medicine stock, calculate total inventory, and identify medicines that require restocking.

Problem Understanding & Context Analysis

Hospitals must maintain sufficient medicine stock to provide uninterrupted patient care. Automated inventory monitoring helps prevent shortages.

Concepts Used in C Programming

- Variables
- Arithmetic Operators
- Relational Operators
- if-else
- for Loop
- break
- continue
- Input/Output Functions

Requirement Analysis

Inputs

- Medicine stock quantity

Outputs

- Total stock
- Restocking status

Solution Design & Justification

The program records medicine stock, calculates the total inventory, identifies low-stock medicines, skips invalid values using continue, and stops if the inventory server fails using break.

Algorithm

1. Start
2. Read medicine stock
3. Skip invalid values
4. Stop if server fails
5. Calculate total stock
6. Display stock status
7. Stop

Complete C Program

```
#include <stdio.h>

int main() {
    int stock, total = 0, i;
    for(i = 1; i <= 10; i++) {
        printf("Enter medicine stock (-1 to stop): ");
        scanf("%d", &stock);
        if(stock == -1)
            break;
        if(stock < 0)
            continue;
        total += stock;
        if(stock < 20)
            printf("Restock Required\n");
    }
    printf("Total Stock = %d\n", total);
    return 0;
}
```

Output

 **Programiz**
C Online Compiler

main.c	Output
<pre>1 #include <stdio.h> 2 int main() { 3 int stock, total = 0, i; 4 for(i = 1; i <= 10; i++) { 5 printf("Enter medicine stock (-1 to stop): "); 6 scanf("%d", &stock); 7 if(stock == -1) 8 break; 9 if(stock < 0) 10 continue; 11 total += stock; 12 if(stock < 20) 13 printf("Restock Required\n"); 14 } 15 printf("Total Stock = %d\n", total); 16 return 0; 17 } 18</pre>	<pre>Enter medicine stock (-1 to stop): 5 Restock Required Enter medicine stock (-1 to stop): 25 Enter medicine stock (-1 to stop): 15 Restock Required Enter medicine stock (-1 to stop): 16 Restock Required Enter medicine stock (-1 to stop): -1 Total Stock = 61 === Code Execution Successful ===</pre>

Explanation of the Code

The program calculates total medicine stock and identifies medicines that need restocking.

Applications

- Hospitals
- Medical Stores

- Healthcare Inventory Systems

Conclusion

The system helps hospitals manage medicine inventory efficiently.

Question 19: Smart Industrial Machine Monitoring System

Title

Smart Industrial Machine Monitoring System

Problem Statement

Develop a C program to monitor machine operating hours, calculate total operating time, and identify machines requiring maintenance.

Problem Understanding & Context Analysis

Industrial machines require regular maintenance to ensure uninterrupted production. Monitoring operating hours helps schedule maintenance efficiently.

Concepts Used in C Programming

- Variables
- Arithmetic Operators
- Relational Operators
- if-else
- for Loop
- break
- continue
- Input/Output Functions

Requirement Analysis

Inputs

- Machine operating hours

Outputs

- Total operating hours
- Maintenance status

Solution Design & Justification

The program records machine operating hours, calculates total operating time, and identifies machines requiring maintenance. Invalid values are skipped using continue, and monitoring stops if the machine controller fails using break.

Algorithm

1. Start
2. Read operating hours
3. Skip invalid values
4. Stop if controller fails
5. Calculate total hours
6. Display maintenance status
7. Stop

Complete C Program

```
#include <stdio.h>

int main() {
    int hours, total = 0, i;
    for(i = 1; i <= 5; i++) {
        printf("Enter machine hours (-1 to stop): ");
        scanf("%d", &hours);
        if(hours == -1)
            break;
        if(hours < 0)
```

```

        continue;
    total += hours;
    if(hours > 1000)
        printf("Maintenance Required\n");
}
printf("Total Operating Hours = %d\n", total);
return 0;
}

```

Output

main.c	Output
<pre> 1 #include <stdio.h> 2 int main() { 3 int hours, total = 0, i; 4 for(i = 1; i <= 5; i++) { 5 printf("Enter machine hours (-1 to stop): "); 6 scanf("%d", &hours); 7 if(hours == -1) 8 break; 9 if(hours < 0) 10 continue; 11 total += hours; 12 if(hours > 1000) 13 printf("Maintenance Required\n"); 14 } 15 printf("Total Operating Hours = %d\n", total); 16 return 0; 17 } 18 </pre>	<pre> Enter machine hours (-1 to stop): 45 Enter machine hours (-1 to stop): 33 Enter machine hours (-1 to stop): 15 Enter machine hours (-1 to stop): 19 Enter machine hours (-1 to stop): -1 Total Operating Hours = 112 === Code Execution Successful === </pre>

Explanation of the Code

The program monitors machine operating hours and identifies machines requiring maintenance.

Applications

- Manufacturing Industries
- Factory Automation
- Predictive Maintenance

Conclusion

The system helps industries improve machine maintenance and reduce downtime.

Question 20: Smart Hotel Room Booking System

Title

Smart Hotel Room Booking System

Problem Statement

Develop a C program to manage hotel room bookings, calculate total rooms booked, and identify room availability.

Problem Understanding & Context Analysis

Hotels require an efficient booking system to manage room reservations and avoid overbooking. Automation improves customer service and booking accuracy.

Concepts Used in C Programming

- Variables
- Arithmetic Operators
- Relational Operators
- if-else
- for Loop
- break
- continue

- Input/Output Functions

Requirement Analysis

Inputs

- Number of rooms requested

Outputs

- Booking status
- Available rooms
- Total rooms booked

Solution Design & Justification

The program checks room availability before confirming bookings. It updates room availability after each booking, ignores invalid requests using continue, and stops processing if the booking server fails using break.

Algorithm

1. Start
2. Read room request
3. Check availability
4. Confirm booking
5. Update available rooms
6. Display booking status
7. Stop

Complete C Program

```
#include <stdio.h>
int main() {
    int rooms = 50, request, booked = 0;
    while(1) {
        printf("Enter rooms required (-1 to stop): ");
        scanf("%d", &request);
        if(request == -1)
            break;
        if(request <= 0)
            continue;
        if(request <= rooms) {
            rooms -= request;
            booked += request;
            printf("Booking Confirmed\n");
        }
        else
            printf("Rooms Not Available\n");
    }
    printf("Total Rooms Booked = %d\n", booked);
    printf("Rooms Available = %d\n", rooms);
    return 0;
}
```

Output



C Online Compiler

main.c	Output
<pre>1 #include <stdio.h> 2 int main() { 3 int rooms = 50, request, booked = 0; 4 while(1) { 5 printf("Enter rooms required (-1 to stop): "); 6 scanf("%d", &request); 7 if(request == -1) 8 break; 9 if(request <= 0) 10 continue; 11 if(request <= rooms) { 12 rooms -= request; 13 booked += request; 14 printf("Booking Confirmed\n"); 15 } 16 else 17 printf("Rooms Not Available\n"); 18 } 19 printf("Total Rooms Booked = %d\n", booked); 20 printf("Rooms Available = %d\n", rooms); 21 return 0; 22 }</pre>	<pre>Enter rooms required (-1 to stop): 20 Booking Confirmed Enter rooms required (-1 to stop): 12 Booking Confirmed Enter rooms required (-1 to stop): 6 Booking Confirmed Enter rooms required (-1 to stop): -1 Total Rooms Booked = 38 Rooms Available = 12 === Code Execution Successful ===</pre>

Explanation of the Code

The program checks room availability, confirms bookings, updates available rooms, and prevents overbooking.

Applications

- Hotels
- Resorts
- Online Booking Systems

Conclusion

The program automates hotel room booking and efficiently manages room availability.